

— XJTUACM —

绪论

西安交通大学 2026 年夏季 ACM 小学期课程



课程简介

- 前置课程：
 - 计算机程序设计。
- 课程内容：
 - 基础算法与数据结构知识（超出我校本科生正常课程范围）。
 - 有一定的难度。

课程简介

- 授课方式：
 - 小学期第 1 周周二至周六、第 2 周周一至周五。
 - 早 9:00 至 12:00 授课，具体时长取决于课程内容，通常在两小时左右。
- 练习题：每天课后布置若干道线上编程题。与当天授课内容强相关。不做练习题完成的强制要求。

课程简介

- 课程交流 QQ 群：
 - 526350936（我的 ACM 之路果然有问题）。
 - 同时，该群也是西安交通大学算法竞赛交流群。
- 课程编程语言：
 - 建议使用 C++ 语言。
 - 若你熟悉 C 语言，在该课程中，同样强烈建议使用 C++ 语言。
 - C 与 C++ 非常相似，C 语言代码略微修改一下，就可以得到等价的 C++ 语言代码。
 - 授课过程以 C++ 为准，其它语言可自行参考其它资料。

课程安排

	周一	周二	周三	周四	周五	周六
第一周		绪论与基础知识	贪心思想	搜索算法	动态规划	分治思想
主讲人		黄万平	陈智	王奕尧	陈弘烜	王奕尧
第二周	初级数据结构	图论基础	数学基础 1 数论、线性代数	数学基础 2 组合数学、博弈论	字符串理论初步	
主讲人	张未艾	李子涵	刘一可	陈儒恒	黄万平	

课程安排

- 0 学分。
- 2026 年 ACM 小学期课程自愿参加。目的不是获取分数，而是提升个人程序设计和算法能力。
- 您可能会在以下场景使用到这些知识与能力：XCPC/蓝桥杯/GPLT天梯赛/CSP认证，求职时遇到的算法题，保研/夏令营机试。
- 不计算课程分数。

绪论与基础知识 主讲人

- 黄万平
- 请于 QQ: 2586647166; 或者邮箱: hwpvector@foxmail.com 联系。
- 特别鸣谢: 陈子谦 齐智 张晓昊
- 上课过程中, 及时反馈
- 绪论与基础知识部分重点内容将用红色标出。

关于作业

- 练习题会在各大 OJ 上。
 - OJ: Online Judging System (在线评测平台)
- 这门课都没有分数。所以：
- 请各位完成练习题的时候，**不要抄袭、不要用大语言模型生成代码**而自己却不理解、直接提交。不要这样欺骗自己已经会了这道题。不要浪费评测资源。
- 请切实地为自己程序设计、算法能力提升而做题。
- 对于算法竞赛初学者而言：**不独立思考题目/作业、不写题目/作业，而只听课等于没学。**

关于 ACM-ICPC

- ACM 国际大学生程序设计竞赛（简称 ACM-ICPC 或 ICPC）是（曾经由）国际计算机协会（ACM）主办的，一项旨在展示大学生创新能力、团队精神和在压力下编写程序、分析和解决问题能力的年度竞赛。
- ACM: Association for Computing Machinery
- ICPC: International Collegiate Programming Contest
- 国内 CCPC（中国大学生程序设计竞赛）与之类似，两者统称为 XCPC。是西安交通大学学生算法竞赛协会主要参加的比赛。



关于 ACM-ICPC

- 比赛期间，每队 3 人使用 1 台计算机在 5 个小时内使用 C / C++ / Java / Python 编写程序解决 8 到 13 个编程问题。
- 每支参赛队伍允许携带纸质资料，允许在小范围内走动，允许与队伍成员之间的交谈和讨论。
- 程序完成之后提交裁判运行，运行的结果会判定为 AC（正确）WA（错误）CE（编译错误）TLE（超时）MLE（超出内存限制）RE（运行错误）或其它结果并立刻被参赛队知晓。

关于 ACM-ICPC

- 每队在正确完成一题后，组织者将在其位置上升起一只代表该题颜色的气球。
- 可以多次提交，在 AC 之前每次错误的提交都将额外产生 20min 的罚时。
- 最终各队伍按照 AC 题数为第一关键字、罚时为第二关键字排行。
- 在前 4 个小时，参赛队伍可以实时查看所有队伍通过的问题和自己的排名。
在最后一个小时，其他队伍的提交结果无法查看，但仍然能查看其他队伍的提交记录、获知自己的提交结果（这一阶段被称为封榜）。

关于 ACM-ICPC (迫真)

	内容纯粹派	内容中立派	内容革新派
	必须是比算法题	只要和算法扯得上关系就行	管他比啥呢
赛制纯粹派	ICPC和CCPC是ACM	数模是ACM	肯德基三人篮球赛是ACM
必须是三人组队参赛			
赛制中立派	OI是ACM	CTF是ACM	星际争霸战队联赛是ACM
只要是比赛就行			
赛制革新派	刷leetcode是ACM	数据结构考试是ACM	睡觉也是ACM
为啥需要是比赛			

西安交通大学程序设计竞赛校赛

- 为激发学生参赛热情，进一步提高我校学生程序设计能力和运用所学知识解决实际问题的能力，培养青年学生的创新意识和团队协作精神，计算机学院、校团委联合学生算法竞赛协会，于每年的 5 月份举办 西安交通大学程序设计竞赛校赛（Xi'an Jiaotong University Programming Contest, XJTUPC）。
- XJTUPC 严格对标 ICPC 核心赛制，聚焦算法设计与实战能力的深度融合，致力于为参赛者打造一个兼具专业性与包容性的成长平台。
- 赛事不设经验壁垒，唯以热爱为舟！无论你是初探代码世界，还是身经百战，这里都将为你开启一段充满挑战与收获的编程之旅！
- 欢迎大家的参赛。

西安交通大学学生算法竞赛校队

- 西安交通大学算法竞赛校队负责组织我校程序设计竞赛选手的训练和选拔。
- 校队主要组织参赛的竞赛包括：国际大学生程序设计竞赛（ICPC）系列比赛、中国大学生程序设计竞赛（CCPC）系列比赛、陕西省大学生程序设计竞赛；
- 校队协助选拔与参赛的竞赛包括：中国高校计算机大赛（CCCC）团体程序设计天梯赛（GPLT）、CCF 大学生计算机系统与程序设计竞赛（CCSP）
- 本次西安交通大学 2026 年夏季 ACM 小学期课程的主讲人均来自西安交通大学学生算法竞赛校队。若想要加入校队，可以等待暑期的选拔比赛。

西安交通大学学生算法竞赛协会、西北多校联合训练

- 西安交通大学程序设计竞赛校队 隶属于 西安交通大学学生算法竞赛协会。
- 西安交通大学学生算法竞赛协会希望给各个阶段的算法竞赛选手，提供稳定的训练环境、讨论环境。
- 西安交通大学学生算法竞赛协会，与西北工业大学、西安电子科技大学、长安大学、陕西师范大学、西北大学等高校开展西北多校联合训练，提升各个阶段的算法竞赛选手的综合水平。
- 西北多校联合训练定期举办比赛，并计算 `rating`（分数）。普及赛：每周一次，线上进行；提高赛：每月一次，线下进行。部分 XCPC 邀请赛的名额分配，将参照该提高赛 `rating`。
- 想参加的选手，可以查看 我的 ACM 之路果然有问题 群群公告。

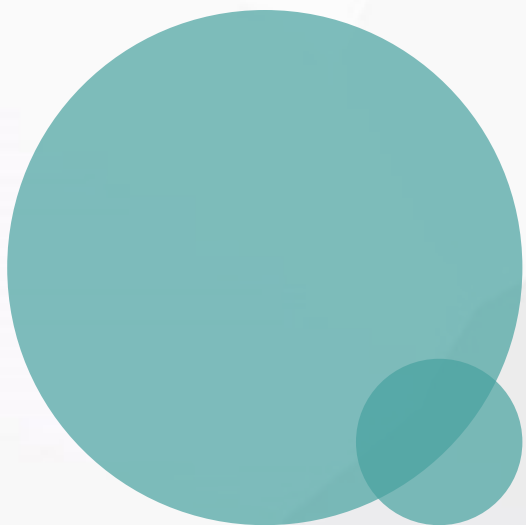
— XJTUACM —

基础知识



课程内容

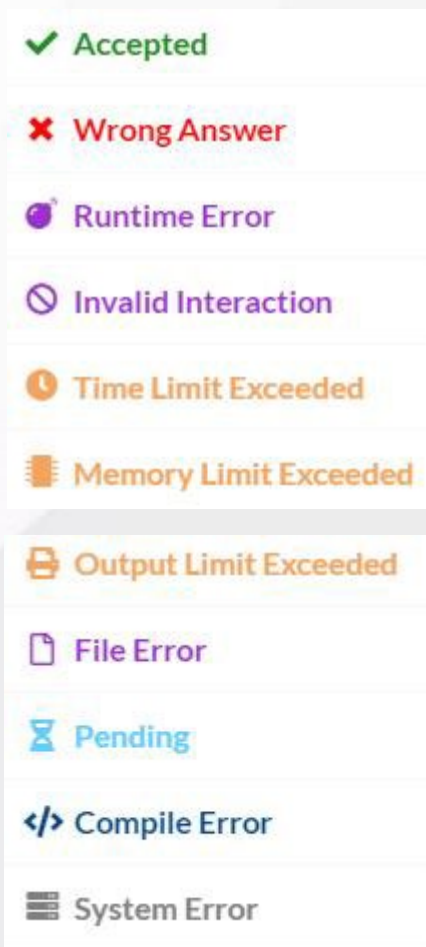
- 一些常识（一）
- 时间复杂度
- 排序算法
- STL 选讲
- 位运算
- 枚举（二进制枚举和折半枚举）
- 一些常识（二）
- 常见代码错误



一些常识（一）

OJ 相关，科学记数法，常用类型，引用

关于OJ



AC 答案正确

WA 答案错误

RE 运行时错误

TLE 运行超时

MLE 运行超内存

OLE 输出超限

CE 编译错误

SE 系统错误

UKE(Unknown Error) 未知错误

关于OJ

每道题均有多个测试点，并且比样例更加强。通过样例并不代表能通过所有数据（讲个笑话我样例过了）。

评测系统会使用事先准备好一些输入数据和相应的输出数据作为测试点，将选手提交的源代码编译后，让选手程序读入输入数据，通过将选手输出与事先准备好的输出比较，来判断选手程序是否正确。

每道题均有一定的时间和空间限制，以及题目中所给出的数据有一定范围。对**所有测试点**必须全部顺利运行，给出正确答案，不超时不超内存，才能通过此题。

上述过程称为黑盒评测。

科学记数法

- xey 表示 $x \times 10^y$ 。
- 例如 $1e5$ 表示 10^5 ， $2e9$ 表示 2×10^9 。
- 在 C++ 中，**xey** 返回的是 **double** 类型

常用类型

- `int` 4 字节 $[-2^{31}, 2^{31} - 1]$, 极大值在 $2.1\text{e}9$
- `long long` 8 字节 $[-2^{63}, 2^{63} - 1]$, 极大值在 $9\text{e}18$
- `unsigned long long` 8 字节 $[0, 2^{64} - 1]$, 极大值在 $1.8\text{e}19$
- `char` 1 字节 $[-128, 127]$
- `double` 8 字节, 约 17 位精度
- `bool` 占用空间 1 字节, 实际只有 1 位

每个字节 8 位, 每一位都是一个 01

C++ 程序

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
int main()
{
    ll x,y;
    cin>>x>>y;
    cout<<x<<' '<<y<<endl;
    return 0;
}
```

`<bits/stdc++.h>`: 万能头, 包含有算法竞赛里几乎所有用到的内容

`using namespace std`: 使用 `std` 命名空间

`typedef`: 为数据类型分配别名。如果你不想打冗长的 `long long`, 就拿 `ll` 代替吧

`cin, cout`: C++ 的标准输入输出方式, 简单易用。虽然速度有点慢。

引用

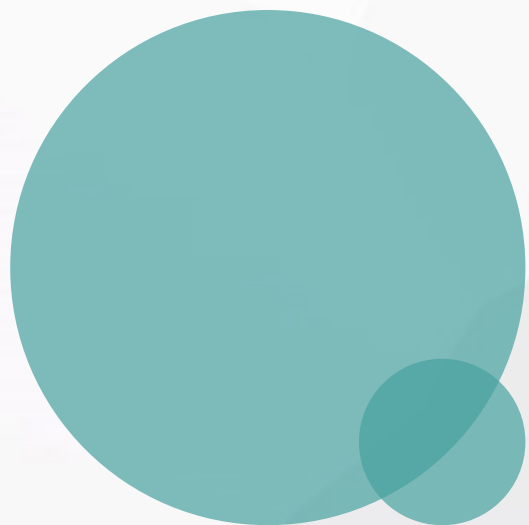
- C++ 的函数默认为值传递，可以在接受参数时加上 & 符号表示引用传递
- 引用，相当于给变量取别名。
- 函数可以通过引用直接访问和修改原始值，而无需创建副本

```
void swap(int a, int b)
{
    int t=b;
    b=a;
    a=t;
}
```

没有任何效果

```
void swap(int &a, int &b)
{
    int t=b;
    b=a;
    a=t;
}
```

成功交换



时间复杂度

时间复杂度

- 如何评价一个算法的运行快慢？

```
bool IsPrime(int x)
{
    for (int i=2;i<=x-1;i++)
        if (x%i==0)
            return false;
    return true;
}
```

- 比如，写出代码跑一下试试？
- 受硬件影响大。
- 使用的用例不同，算法效率可能天差地别。例： $x=998244353$ ， $x=998244354$
- 通常情况下，我们希望在写程序之前就能大致确定算法效率。

时间复杂度

- 如何评价一个算法的运行快慢？

```
bool IsPrime(int x)
{
    for (int i=2;i<=x-1;i++)
        if (x%i==0)
            return false;
    return true;
}
```

- 记 $T(n)$ 是输入规模为 n 时，算法所需要执行的基本操作数（如加减乘除、判断、返回、简单类型赋值、数组访问等等，每执行一次，算一次操作）。那么，我们可以通过计算得出算法所需时间的一个上界表达式。这就是时间复杂度。
- 对于以上算法，我们可以得知， $T(n) \leq 3n$ 。

时间复杂度

- 好了，你已经学会了，快来分析以下算法的效率吧！

```
int main()
{
    scanf("%s", s+1);
    for(char i=0; i<127; ++i) if(('a'<=i&&i<='z') || ('A'<=i&&i<='Z') || ('0'<=i&&i<='9')) id[i]=++tot;
    n=strlen(s+1);
    for(int i=1; i<sig; ++i) t[i]=0;
    for(int i=1; i<=n; ++i){for(int j=1; j<sig; ++j) pre[i][j]=t[j]; t[id[s[i]]]=i;}
    for(int i=1; i<sig; ++i) t[i]=n+1;
    for(int i=n; i; --i){for(int j=1; j<sig; ++j) nxt[i][j]=t[j], nxs[i][j]=nxs[i+1][j]+(id[s[i]]==j); t[id[s[i]]]=i;}
    for(int i=n; i; --i){int m=id[s[i]];
        for(int j=1; j<sig; ++j){if(j==m) continue; g[m][pre[i][j]]+=nxs[i+1][j];}}
    for(int i=1; i<=n; ++i) for(int j=1; j<sig; ++j) nxs[i][j]=nxs[i-1][j]+(id[s[i]]==j);
    for(int i=1; i<=n; ++i) {long long tmp=1LL*i*(i-1)/2;
        for(int j=1; j<sig; ++j) tmp-=1LL*nx[s][i][j]*(nx[s][i][j]-1)/2; p[i]=tmp%mod;}
    for(int j=1; j<sig; ++j){
        for(int i=n; i; --i) inc(g[j][i], g[j][nxt[i][id[s[i]]]]); for(int i=n; i; --i) inc(g[j][i], g[j][nxt[i][id[s[i]]]]);}
    for(int i=1; i<=n; ++i){
        if(i<=2) continue; if(i>n-4+1) continue; int m=id[s[i]];
        for(int o=1; o<sig; ++o)
        {
            int f=g[m][nxt[i][o]];
            int t=dic(add(p[i-1], mul(nxs[i-1][o], nxs[i-1][m])), add(mul(nxs[i-1][m], i-1-nxs[i-1][m]), mul(nxs[i-1][o], i-1-nxs[i-1][o])));
            inc(ans, mul(f, t));
        }
    }
    printf("%d\n", ans);
    return 0;
}
```

时间复杂度

- 对于复杂的算法而言，计算精确的语句执行次数是不切实际的。我们也并不关心具体的运算频次是多少，而只需要估算大数据下的大致量级（因为题目中、计算机通常是需要处理大数据的）。
- 若 $T(n) = n^2 + 7n + 1919810\log_2 n$ ，那么，当 $n \rightarrow +\infty$ 时，哪一项决定了整体的大小呢？
- 例如，当 $n = 114514$ 时，有：
- $n^2 = 13,113,456,196$
- $7n = 801,598$
- $1919810\log_2 n = 32,262,723$
- 也即，当 n 很大的时候，表达式的值实际上主要取决于增长速度最快的一项！

时间复杂度

- 我们记：
- $\Theta(g(n)) = \{f(n) | \exists c_1, c_2, n_0 > 0, s.t. \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\}$
- 即 $\Theta(g(n))$ 是增长速度与 $g(n)$ 同阶的函数集合。
- $O(g(n)) = \{f(n) | \exists c_2, n_0 > 0, s.t. \forall n \geq n_0, 0 \leq f(n) \leq c_2 g(n)\}$
- 即 $O(g(n))$ 是增长速度比 $g(n)$ 同阶或低阶的函数集合。
- $\Omega(g(n)) = \{f(n) | \exists c_1, n_0 > 0, s.t. \forall n \geq n_0, c_1 g(n) \leq f(n)\}$
- 即 $\Omega(g(n))$ 是增长速度比 $g(n)$ 同阶或高阶的函数集合。

时间复杂度

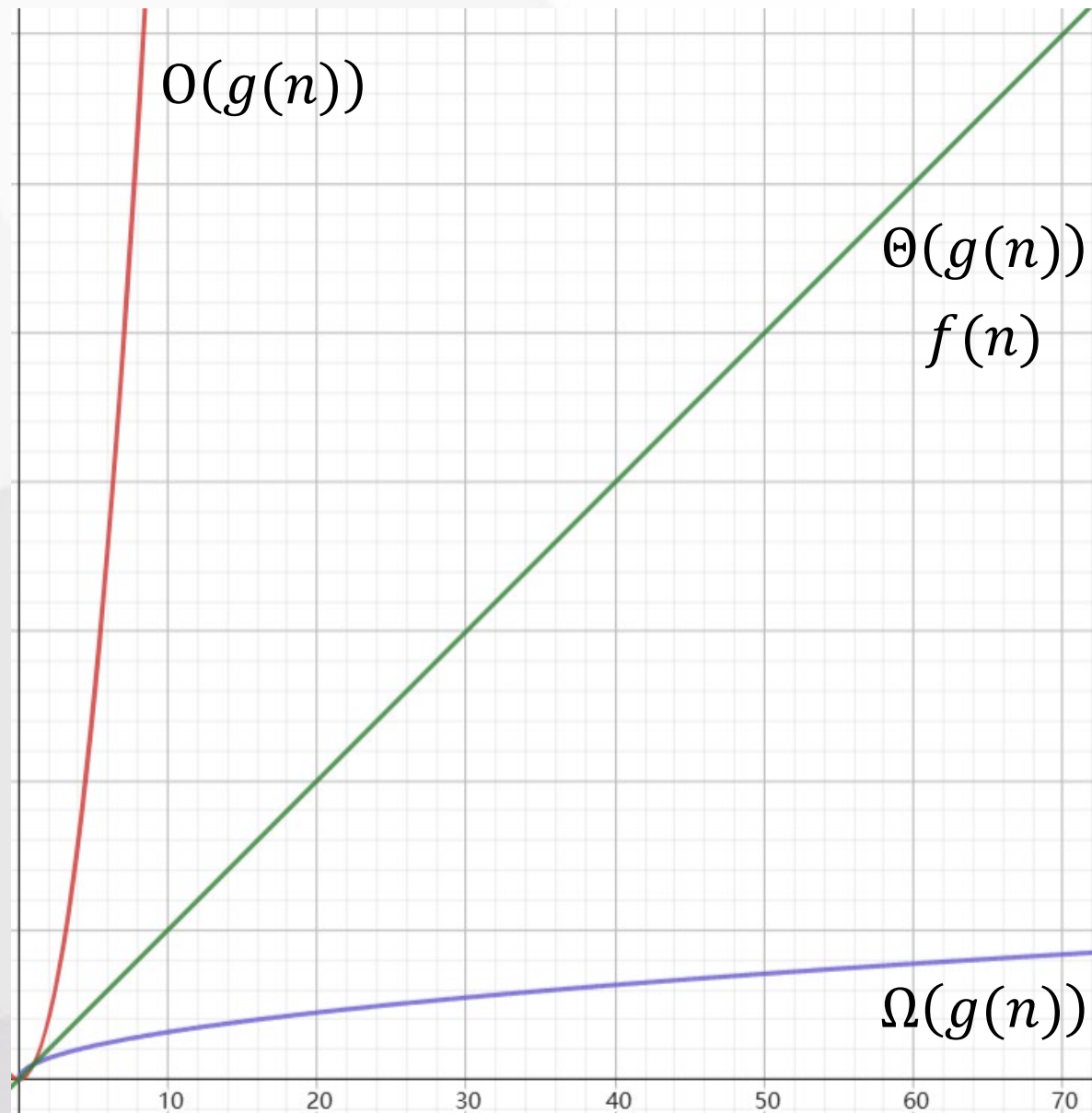
例如，若 $T(n) = n^2 + 7n + 1919810\log_2 n$

则 $T(n) = \Theta(n^2) = O(n^2)$

注意此处的等号暗指集合的包含关系

如果 $T(n) = O(n^2)$ ，那么 $T(n) = O(n^3)$?

- 大 O 记号具有可以增长速度变快的特性，换句话说，我们只是找到了一个上界，而不一定是紧界
- 同样，大 Ω 符号具有增长速度变慢的特性，这只是一个下界，不一定是紧界



时间复杂度

- 一个程序的时间复杂度 $T(n)$ ，不一定存在紧界。它可能在最坏情况下到达 $O(n^2)$ ，而在最好情况下到达 $O(n \ln n)$ 。
- 之前的例子：

```
bool IsPrime(int x)
{
    for (int i=2; i<=x-1; i++)
        if (x%i==0)
            return false;
    return true;
}
```


时间复杂度

- 一个程序的时间复杂度 $T(n)$ ，不一定存在紧界。它可能在最坏情况下到达 $O(n^2)$ ，而在最好情况下到达 $O(n \ln n)$ 。
- 通常情况一下，一道算法竞赛题会有一些的时间限制。而题目的数据在不加额外说明的情况下，会默认将你所写的算法的时间复杂度卡到最坏情况。
- 也就是说，我们通常会考虑的是，一个算法的上界而非紧界、下界
- 需要注意，课程中如果不加以说明，当我们称某算法的时间复杂度为 $O(g(n))$ 时，我们实际上是在说，在最坏情况下，该算法的时间复杂度是 $\Theta(g(n))$ 。在做题时，同样需要对时间复杂度最坏情况的紧界进行计算。

时间复杂度

- 如果 $O(f_1(n))$ 比 $O(f_2(n))$ 的增长速度快, 则记为 $O(f_1(n)) > O(f_2(n))$
- 若 $c_1 > c_2 > 0$, 则 $O(n^{c_1}) > O(n^{c_2})$
- 若 $c_1 > 0$, 则 $O(n!) > O(3^n) > O(2^n) > O(n^c) > O(\ln n) > O(\ln \ln n) > \dots$
- $\forall a \neq 1, \log_a n = O(\ln n)$
 - 由换底公式可知, 任何对数函数无论底数为何, 都有相同的增长速度
 - 在时间复杂度中通常不写对数函数的底数, 而是以 $\ln n$ 或 $\log n$ 替代
 - 在计算机科学中, 我们默认, 所有 $\log$ 函数, 以2为底

时间复杂度

- 如何分析一个程序的时间复杂度？
- $f_1(n) \times f_2(n) = O(f_1(n) \times f_2(n))$
 - 通常情况下，数循环几层循环就是几次的复杂度
 - 当然，不一定是紧界，反例很多
 - 最重要的是计算算法的基本操作运行了多少次
- $c_1 \cdot O(g(n)) + c_2 = O(g(n))$ ，其中 $c_1, c_2 > 0$ 为常数
 - 忽略常量，也即，一段代码执行一次还是十次不影响复杂度
 - 所有和输入规模无关的量都被视作常量，计算复杂度时可当作 1 处理
- $f_1(n) + f_2(n) = O(\max(f_1(n), f_2(n)))$
 - 多个部分复杂度取最大值

- 分析如下程序的时间复杂度:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++)  
        sum += i*j;  
}
```

```
for (int i = 1; i <= n; i += i) {  
    for (int j = 1; j <= n; j++)  
        sum += i*j;  
}
```

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j+=i)  
        sum += i*j;  
}
```

```
for (int i = 1; i <= n; i++) {  
    for (int j = i; j <= n; j++)  
        sum += i*j;  
}
```

```
for (int i = 1; i <= n; i += i) {  
    for (int j = i; j > 0; j--)  
        sum += i*j;  
}
```

(常用) 调和级数求和:

$$\sum_{i=1}^n \frac{1}{n} = \ln n + \gamma + O(1/n)$$

(或者通过积分计算增长速度)

时间复杂度

- 计算机每秒能进行的运算次数通常在 10^8 数量级。
- 在编写一个程序之前，可以先计算一个算法的时间复杂度。
- 通过代入题目中的数据范围，计算该算法在最差情况下的运算次数。
- 于是，便可以估算在最差情况下程序所需的运行时长，来判断自己的程序是否会运行超时。
- 一般的算法竞赛题不会卡算法的常数。

时间复杂度表

时间复杂度	10	10^2	10^3	10^4	10^5	10^6
n	10	100	1000	1万	10万	100万
$n \log n$	33	664	9965	13万	166万	1993万
$n(\log n)^2$	255	8790	1.76万	289万	4269万	5.88亿
$n \sqrt{n}$	31	1000	3.16万	100万	3162万	10亿
n^2	100	1万	100万	1亿	100亿	1万亿
n^3	1000	100万	10亿	1万亿	10^{15}	10^{18}
2^n	1000	10^{30}	<u>inf</u>	<u>inf</u>	<u>inf</u>	<u>inf</u>
$n!$	363万	<u>inf</u>	<u>inf</u>	<u>inf</u>	<u>inf</u>	<u>inf</u>

递归函数的时间复杂度

- 画出递归树

斐波那契数列 $f_1 = 1, f_2 = 1, f_n = f_{n-1} + f_{n-2}$

求斐波那契数列第 n 项的程序的时间复杂度

```
int fib(int n) {  
    if (n == 1) return 1;  
    if (n == 2) return 1;  
    return fib(n - 1) + fib(n - 2);  
}
```

思考：每个元素重复计算了多次，导致了时间复杂度较高。如何修改？

递归函数的时间复杂度

记录每一个数是否曾经计算过。

当递归到某一个数时，先判断曾经是否计算过，如果是则直接输出结果；否则，计算当前的数，并把当前的数记录为计算过，同时记录所得的值。这样一个算法的时间复杂度是多少？（该算法被称为记忆化搜索）

```
int fib(int n) {  
    if (n == 1) return 1;  
    if (n == 2) return 1;  
    if (vis[n] == 1) return fib[n];  
    vis[n] = 1;  
    fib[n] = fib(n - 1) + fib(n - 2);  
}
```


求以下函数在最坏情况下的时间复杂度（另类的二分查找算法）。

```
int binarySearch(int arr[], int target, int l, int r) {  
    if (r < l) return -1;  
    for (int i = l; i <= r ; ++i) cout << arr[i];  
    int mid = l + (r - l) / 2;  
    if (arr[mid] == target) return mid;  
    else if (array[mid] > target)  
        return binarySearch(arr, target, l, mid - 1);  
    else  
        return binarySearch(arr, target, mid + 1, r);  
}
```

在主函数中调用 `binarySearch(arr,114514,0,n-1);`

求以下函数的时间复杂度（另类的建立线段树算法）。

```
void build(int a[], int sum[], int l, int r, int k) {  
    if (l == r) { sum[k] = a[l]; return; }  
    int mid = l + ((r - l) / 2);  
    build(a, sum, l, mid, k * 2);  
    build(a, sum, mid + 1, r, k * 2 + 1);  
    sum[k] = 0;  
    for (int i = l; i <= r; ++i) sum[k] += a[i];  
}
```

在主函数中调用 `build(a, sum, 1, n, 1);`

递归函数的时间复杂度

对于部分递归算法，我们常常有这样的结构：

```
Def solve(problem) { // 一个解决问题的函数
    solve without recursion(); // 通过非递归的形式预处理问题
    for subproblem in problem // 将问题分为几个相同规模的子问题
        solve(subproblem); // 利用递归调用子问题
    merge subproblem without recursion();
    // 通过非递归的形式，合并子问题得到的结果
}
```

递归函数的时间复杂度

我们可以用一种表示方式来概括这些结构的算法：

对于一个规模为 n 的问题，我们把它分为 a 个子问题，每个子问题的规模为 $\frac{n}{b}$ 。
同时，我们也知道当规模 $n < b$ 时问题的答案，可以 $O(1)$ 得出。

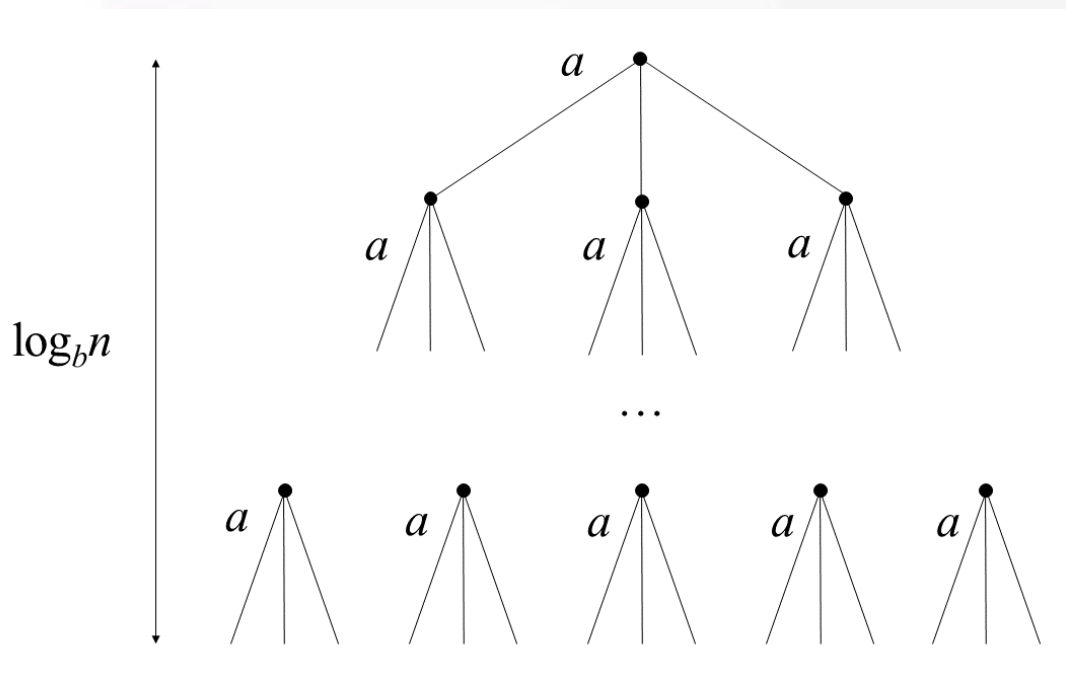
那么这种方法的时间复杂度 $T(n)$ 有如下递归关系：

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

其中， $a \geq 1, b > 1$ 为常数， $\frac{n}{b}$ 指 $\lfloor \frac{n}{b} \rfloor$ 或 $\lceil \frac{n}{b} \rceil$ ， $f(n)$ 为创造这些递归关系、将这些子问题结果整合所需的操作次数。

递归函数的时间复杂度

对于这种递归算法，我们可以建立这样的递归树：



树高为 $\log_b n$ ，树的第 i 层有 a^i 个节点，
每个节点的问题规模为 $\frac{n}{b^i}$ 。

这棵树有 $a^{\log_b n} = n^{\log_b a}$ 个叶子节点。

因此这种方法的复杂度也可以表示为：

$$T(n) = \Theta(n^{\log_b a}) + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$$

Master 定理

主定理（Master定理）解决了此类问题的时间复杂度。

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & f(n) = O(n^{\log_b(a)-\epsilon}), \epsilon > 0 \\ \Theta(f(n)) & f(n) = \Omega(n^{\log_b(a)+\epsilon}), \epsilon \geq 0 \\ \Theta(n^{\log_b a} \log^{k+1} n) & f(n) = \Theta(n^{\log_b a} \log^k n), k \geq 0 \end{cases}$$

Master 定理

主定理（Master定理）解决了此类问题的时间复杂度。

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

（非严格形式）记 $c = \log_b a$ ，且 $f(n) = n^d \log^k n \dots$ ，那么：

$$T(n) = \Theta(n^c), \quad c > d$$

$$T(n) = \Theta(f(n)), \quad c < d$$

$$T(n) = \Theta(f(n) \log^{k+1} n), \quad c = d$$

$T(n)$ 表示输入规模为 n 时, 算法的时间复杂度。求下列递归函数的时间复杂度。

$$T(n) = 2 T\left(\frac{n}{2}\right) + O(1)$$

$$T(n) = T\left(\frac{n}{2}\right) + O(\log n)$$

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$T(n) = 4 T\left(\frac{n}{2}\right) + O(n \ln n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 3 T\left(\frac{n}{2}\right) + O(n)$$

(Master定理) 记 $c = \log_b a$, 且 $f(n) = n^d \log^k n \dots$, 那么:

$$T(n) = \Theta(n^c), \quad c > d$$

$$T(n) = \Theta(f(n)), \quad c < d$$

$$T(n) = \Theta(f(n) \log^{k+1} n), \quad c = d$$

求以下函数在最坏情况下的时间复杂度（另类的二分查找算法）。

```
int binarySearch(int arr[], int target, int l, int r) {  
    if (r < l) return -1;  
    for (int i = l; i <= r ; ++i) cout << arr[i];  
    int mid = l + (r - l) / 2;  
    if (arr[mid] == target) return mid;  
    else if (array[mid] > target)  
        return binarySearch(arr, target, l, mid - 1);  
    else  
        return binarySearch(arr, target, mid + 1, r);  
}
```

在主函数中调用 `binarySearch(arr,114514,0,n-1);`

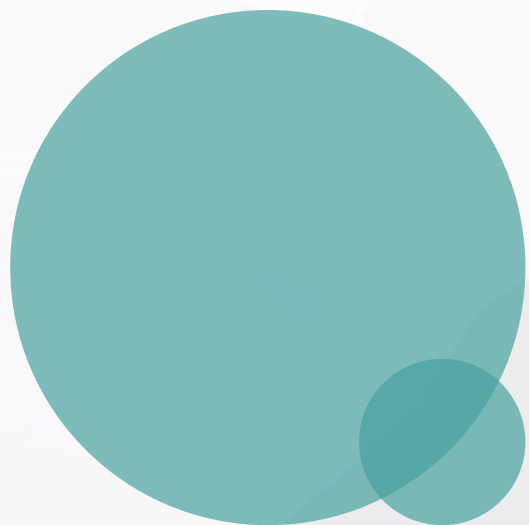
求以下函数的时间复杂度（另类的建立线段树算法）。

```
void build(int a[], int sum[], int l, int r, int k) {  
    if (l == r) { sum[k] = a[l]; return; }  
    int mid = l + ((r - l) / 2);  
    build(a, sum, l, mid, k * 2);  
    build(a, sum, mid + 1, r, k * 2 + 1);  
    sum[k] = 0;  
    for (int i = l; i <= r; ++i) sum[k] += a[i];  
}
```

在主函数中调用 `build(a, sum, 1, n, 1);`

空间复杂度

- 空间，或是说申请的内存大小，或者说你开的数组大小。
- 一般算法竞赛题目不会卡你的空间大小。这里不过多介绍。
- 空间复杂度，就是你开的数组大小，随着 n 或者其他自变量的同阶增长速度。



排序算法

冒泡排序

- 思想：排序 n 趟，每趟找到一个最小的元素将之移动到开头。

```
void bubble_sort (int arr[], int len)
{
    for (int i = 0; i < len; ++i) // 排序 n 趟
        for (int j = len - 1; j > i; --j)
            if (arr[j] < arr[j - 1])
                swap (arr[j] , arr[j - 1]);
}
```

- 例子： 3 1 2 0 5 4
- 时间复杂度： $O(n^2)$ 。很慢的排序算法。

快速排序

- 思想：对于乱序数组 A ，选取一个轴值 x （通常为 $A[1]$ ），把比 x 小的值放在左边，把比 x 大的值放在数组右边。然后递归调用上述过程，再对左边的这些数和右边的这些数分别排序。
- 例子：
 - 将 $[3\ 1\ 2\ 0\ 5\ 4]$ 排序。
 - $[3\ 1\ 2\ 0\ 5\ 4]$
 - $[1\ 2\ 0]\ 3\ [5\ 4]$
 - $[[0]\ 1\ [2]]\ 3\ [[4]\ 5]$
 - $0\ 1\ 2\ 3\ 4\ 5$
- 实现： `std::sort`（见 STL 部分）

快速排序的时间复杂度

设数组的元素个数为 n

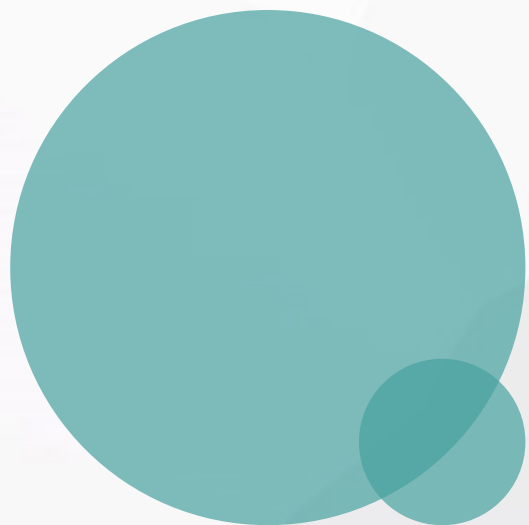
如果对于每一个递归调用，都有 $|l - r| = 1$ ，那么该算法的时间复杂度为多少（此时为最优情况）？

如果对于每一个递归调用，都存在一个 $l = 1$ 或是 $r = 1$ ，那么该算法的时间复杂度为多少（此时为最差情况）？

可以通过放缩证明（<https://www.cnblogs.com/LzyRapx/p/9565827.html>），快速排序的期望复杂度为 $O(n \log n)$ 。这里不多赘述。

桶排序

- 桶：以值为下标建立数组记录信息。
- 数组：1 2 4 3 4 2 3 4 4 4
- 桶：1 2 2 5（1 的元素个数有 1 个，2 有 2 个，3 有 2 个，4 有 5 个）
- 将元素插入桶中。按顺序扫描桶，取出元素。
- 对于值域范围为整数、值域范围较小的数组，有较优秀的运行速度。
- 设值域区间大小为 V ，数组的元素个数为 n ，则复杂度为 $O(n + V)$



STL选讲

大幅简化编程难度

sort

- `sort` 函数：对数组进行排序。
 - 内部实现采用内省排序，当快速排序递归到一定层数后进行堆排序
 - 避免了快速排序的时间复杂度可能退化成 $\Theta(n^2)$ 。
 - 时间复杂度稳定为 $\Theta(n \log n)$ 。

sort

- `sort(begin, end, cmp)`
- `begin` 为排序数组的 第一个元素的地址
 - 数组名 `arr` 即为一个指针常量，可以得到数组第一个元素 (`arr[0]`) 的地址
- `end` 为排序数组的 最后一个元素的下一个元素的地址
 - 换言之，`sort` 的数组地址的传入是左闭右开的
 - 例如，`sort(arr, arr+n)` 是对 `arr[0]`, `arr[1]`, ..., `arr[n-1]` 进行排序
 - `sort(arr+1, arr+n+1)` 是对 `arr[1]`, `arr[2]`, ..., `arr[n]` 进行排序
- `cmp` 是 `sort` 函数的排序规则
 - 可以不写，默认是从小到大排序的
 - 如果需要从大到小可以换成 `greater<T>()`，`T` 为排序数组的变量类型
 - 如果我们需要按照其他的排序规则，我们可以自定义一个 `bool` 类型的函数来传入

排序规则

- `cmp` 函数传入两个量 `T x`, `T y`, 其中 `T` 表示排序数组的变量类型; 同时 `cmp` 函数返回一个 `bool` 类型
- 对于最终的排序结果, 对于任意两个在排序范围内的正整数 `i, j` (`i < j`), 都满足
$$\text{cmp}(\text{arr}[i], \text{arr}[j]) == 1 \text{ or } \text{arr}[i] == \text{arr}[j]$$
- `cmp` 函数也可以理解为排序后的结果, 按照 `cmp` 的顺序进行排列

一个例子: 从大到小排序

```
bool cmp(int x, int y) {  
    return x > y;  
}  
  
int n = 5;  
int arr[M] = {1, 5, 4, 2, 3};  
sort(arr, arr + n, cmp);  
// sort(arr, arr + n, greater<int>());
```

排序规则

- `cmp` 必须满足：
 - 反自反性： $\text{cmp}(x, x)$ 恒等于 0
 - 反对称性：对于 $x \neq y$ ，满足 $\text{cmp}(x, y)$ 或 $\text{cmp}(y, x)$ 有且仅有一个为 1
 - 传递性：若 $\text{cmp}(x, y) = \text{cmp}(y, z) = 1$ ，则 $\text{cmp}(x, z) = 1$
 - 如果上述的性质不满足，可能会导致程序 RE（运行时错误）或是 TLE（超时）或是 WA（答案错误）
 - 如果 $\text{cmp}(x, y) = \text{cmp}(y, x) = 0$ ，则 x, y 满足什么关系？

sort

- 如果要按数组的绝对值从小到大排序，`cmp` 函数应当怎么写？
- 假如有一个结构体，代表一只 ACM 队伍所解出的题数和所用的罚时。

```
struct team {int solveproblem, int time;};
```

你需要对 `team p[3], p[4], p[5], ..., p[n-2]` 进行排序

排序规则为：以 `solveproblem` 为第一关键词，`solveproblem` 越大的，排序后下标越靠前；以 `time` 为第二关键词，当两个 `team x, team y` 的 `solveproblem` 相同时，`time` 越小的，排序后下标越靠前。

应当怎么做？

pair

- 两个元素的组合。可以少打一个结构体。
 - `pair<int,double> p;`
 - `pair<pair<int,double>,pair<int,double>> pp;`
- 访问第一个元素: `p.first`
- 访问第二个元素: `p.second`
- 将两个元素组合成一个 `pair` 类型: `make_pair(v1,v2)`
- 访问 `pair<pair<T,T>,pair<T,T>>` 类型的第一个 `pair` 的第二个元素:
`pp.first.second`
- `pair` 的比较 (`pair<int,double> p1,p2; p1>p2`): 先比较第一个, 如果相同则比较第二个 (字典序)
- 上述操作的时间复杂度均为 $O(1)$

stack

- 栈：后进先出的数据结构，像一群人进入一个非常狭窄的山洞
- 最后进来的元素称为栈顶元素
- 实现时，只需维护数据数组与栈顶指针



栈的典型例子：汉诺塔

stack

- 以 T 类型创建一个栈，创建时栈为空： `stack<T> s;`
- 往栈中加入一个元素 x ： `s.push(x)`
- 删除栈顶元素（注意，返回值为 `void`）： `s.pop()`
- 查询栈顶元素： `s.top()`
- 查询栈的大小： `s.size()`
- 检查栈是否为空（如果为空返回 `1`，非空返回 `0`）： `s.empty()`
- 将栈清空： `s.clear()`
- 上述操作的时间复杂度均为 $O(1)$

stack

输出栈中的所有元素，并清空栈：

```
while(!s.empty()) {  
    cout << s.top() << endl;  
    s.pop();  
}
```

queue

- 队列：先进先出的数据结构，像一群人排队打饭
- 最先进来的元素称为队首元素，最后进来的元素称为队尾元素

queue

- 以 T 类型创建一个队列，创建时队列为空： `queue<T> q;`
- 往队列中加入一个元素 x ： `q.push(x)`
- 删除队首元素（注意，返回值为 `void`）： `q.pop()`
- 查询队首元素： `q.front()`
- 查询队列的大小： `q.size()`
- 检查队列是否为空（如果为空返回 `1`，非空返回 `0`）： `q.empty()`
- 将队列清空： `q.clear()`
- 上述操作的时间复杂度均为 $O(1)$

queue

输出队列中的所有元素，并清空队列：

```
while(!q.empty()) {  
    cout << q.front() << endl;  
    q.pop();  
}
```

vector

- 动态数组，适用于预先不知道开多大的数组，或者想要轻快简便传参的情况
- 可以快速延长数组

vector

- 以 `int` 类型创建一个动态数组，创建时动态数组为空：`vector<int> v;`
- 以 `int` 类型创建一个动态数组，并预分配 10 的大小：`vector<int> v2(10);`
- 往动态数组尾部加入一个元素 `x`：`v.push_back(x)`
- 在动态数组尾部删除一个元素：`v.pop_back()`
- 查询动态数组的大小：`v.size()`
- 访问动态数组下标为 `i` 的元素（默认下标从 0 开始）：`v[i];`
- 将动态数组清空：`v.clear()`

- 除预分配大小外，上述操作的时间复杂度均为 $O(1)$

迭代器

- 迭代器是一种检查容器内元素并遍历元素的数据类型，通常用于对 C++ 中各种容器内元素的访问。不同的容器有不同的迭代器
- 可以将迭代器理解为指针，用法类似
- 迭代器的数据类型名：容器名 + `::iterator`
 - `vector<int>::iterator`
 - `vector<long long>::iterator`
 - `vector<pair<int,double>>::iterator`
- `vector`, `string`, `map`, `set` 等容器都支持迭代器
- `stack`, `queue`, `priority_queue` 不支持迭代器

迭代器

```
vector<int> v; vector<int>::iterator it1,it2;
```

- 指向容器第一个元素的迭代器: `v.begin()`
- 指向容器最后一个元素的下一个位置的迭代器: `v.end()`
- 访问迭代器所指向的元素: `int x=*it1`
- 移动迭代器至后一个元素或前一个元素: `it1++`, `++it1`, `it1--`, `--it1`
- 移动迭代器至后 `x` 个元素或前 `x` 个元素: `it + x`, `it - x`
 - 只有 `vector`, `string`, `deque` 迭代器支持
- 比较两个迭代器的位置关系: `it1<=it2`, `it1>it2`, `it1==it2 ...`

auto

在 C++11 以上，如果在定义变量时对变量进行赋值，赋值类型已知，关键词 `auto` 可以自动补全变量类型

在变量类型极长的情况下，可以节省打出变量类型的时间：

```
auto a=1;
```

```
auto s="XJTUICPC";
```

```
vector<int> v;
```

```
auto it1=v.begin(),it2=v.end()
```

vector

访问动态数组中的全部元素：

```
for (int i = 0; i < vc.size(); ++i)
    v[i];
```

```
for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
    *it;
```

```
for (auto it = v.begin(); it != v.end(); ++it) // 需要 C++11 以上
    *it;
```

```
for (auto i:v) // 范围 for, 需要 C++11 及以上
    i;
```

vector

对动态数组中的全部元素进行从小到大的排序：

```
sort(v.begin(), v.end());
```

string

- 字符串，某种意义上可以认为是 `vector<char>`
- `vector` 的大部分操作（`v.push_back(x)`，`v.size()`，`v.clear()`，`v[i]` 等），在 `string` 上都可以进行
- 同时，`string` 类型又存在新的操作

string

- 创建两个字符串，创建时字符串为空： `string s1,s2;`
- 将字符串 `s1` 赋值为 “XJTUICPC”： `s1=“XJTUICPC”;`
- 读入一个字符串（读入至空格或换行符停止读入）： `cin>>s2;`
- 读入一个字符串（读入至换行符停止读入）： `getline(cin,s2);`
- 将两个字符串合并为一个字符串： `s1+s2`
 - 例如： `string s1=“XJTU”, s2=“ICPC”, s3=s1+s2`
 - 则： `s3==“XJTUICPC”`

map

- 映射、关联数组
- 以键值对的方式存储
 - 键值对: `pair<T1, T2> p`
 - `p.first` 称为键
 - `p.second` 称为值
- 不允许键重复, 并根据键自动排序
- 可以使用类似数组的方式去操作
- 类似 Python 的字典, 内部采用红黑树维护

map

- 键值对可以理解为 一一映射的关系
- 可以将 **map** 看一个数组，可以是任意类型下标的数组，不过速度比较慢；数组下标为键，数组存储的值为值
- 以键值对类型为 `pair<int, double>` 创建一个关联数组，创建时关联数组为空：
`map<int, double> mp;`
- 向关联数组中插入键值对 `(14, 0.1)`: `mp[14] = 0.1;`
 - 如果之前插入过键 `14`，则将之前的键值对替换为 `(14, 0.1)`
- 查询关联数组键 `114514` 对应的值: `mp[114514]`
- 将关联数组 `114514` 对应的值自增一: `mp[114514]++;`
- 查找键所对应的迭代器（不存在则返回 `mp.end()`）: `auto it=mp.find(114514)`
- 删除迭代器所指向的键值对、删除键所对应的键值对: `mp.erase(it), mp.erase(14)`
- 清空关联数组: `mp.clear();`
- 访问、插入、删除、查询关联数组中的键值对的复杂度均为 $O(\log n)$ ， n 为关联数组大小

map

- 遍历关联数组（注意关联数组采用键值对存储，按照键有序排列）

```
for (auto it = mp.begin(); it != mp.end(); ++it) // 需要 C++11 以上
    *it; // *it 是一个 pair 类型，第一个数表示键，第二个数表示值
    it -> first, it -> second; // first 表示键，second 表示值
for (auto p:mp) // 范围 for，需要 C++11 及以上
    p.first, p.second; // p.first 表示键，p.second 表示值
for (auto [key, value]:mp) // 结构化绑定，需要 C++17 及以上
    key, value; // key 表示键，value 表示值
```

其他

- `priority_queue` 优先队列
- `set/multiset` 集合
- `unordered_map` 哈希映射
- `deque` 双端队列
- `bitset` 位向量
- `list` 链表
- `unique` 去重
- `lower_bound/upper_bound` 二分查找